Agentic PMS: developer guide

Contents

1. What the product is
2. Architecture at a glance
3. Repository map
4. Running it locally
5. How a request travels
6. Tenancy
7. Sign-in and sessions
8. Permissions and whose rows you see
9. Database and migrations
10. Server: core services
11. Server: the performance module
12. Server: people, engagement and agentic
13. AI features
14. Notifications, mail and files
15. Background jobs
16. Frontend
17. Tests
18. CI and deployment
19. Recipes: making common changes
20. House rules
21. Known gaps and gotchas
22. Glossary

This guide is for a developer joining the project. It covers what the system is made of, how a request travels through it, where each feature lives, and how to run, test, change and ship it. Every statement names the file it comes from, so you can check it against the code.

It covers the code, not the business rules. For "why does the screen do that", read **LOGIC-BOOK.md** (the rules, section by section). For what each user does, read the manuals (`MANUAL-EMPLOYEE.md`, `MANUAL-MANAGER.md`, `MANUAL-HR.md`). For hosting, read `DEPLOY.md`, `deploy/README.md` and `DEPLOY-GITHUB-ACTIONS.md`.

Written against build `211b79e` (20 Oct 2026). The live build is at `https://pms.agentichumans.in/api/v1/health`.

1. What the product is

Agentic PMS is an employee performance management system. It was extracted as a product from a larger HR platform (the "AH platform"). Each client gets their own deployment: one server, one database, one tenant.

What it does, by area:

Area	What people do	Server module
Performance cycle	Set KRAs (objectives) and their weights, mid-year review, self appraisal, manager evaluation, HOD review, HR calibration, publish a final rating	`performance`
Around the cycle	Connects (1:1 check-ins), minutes of meetings, competencies, growth and career plans, timesheet coverage per KRA, Improvement Plans (PIP), 9-box grid, Super 50 (top-talent watchlist and benefits), increment simulation, closure letters	`performance`
People	Rewards and recognition (RnR) with an approval chain, events and CSR, queries to HR or colleagues, career pathing matrix, onboarding tracker	`people`
Engagement	Surveys (pulse, lifecycle, exit…), anonymity, insights	`engagement`
AI drafts	Draft text for reviews, connects, minutes, career plans. Never ratings.	`agentic` (plus `core/ai.js`)

Roles: employee, manager, HOD (head of department), HRBP (HR business partner, limited to a remit), HR, HR Ops, Super Admin. The menu has a section per role (Self, Manager, HOD, HRBP, HR, HR Ops) and shows only the sections a person may open.

2. Architecture at a glance



- **Stack:** Node 22, Express 4, `pg`, `jsonwebtoken`, `bcryptjs`, `multer`, `exceljs`, `pdfkit`, `nodemailer` (`server/package.json`). React 18, react-router 6, Vite 5, Tailwind 3, lucide-react icons (`frontend/package.json`).
- **No ORM.** Handlers write SQL directly with `db.query(text, params)` (`server/core/db.js`).
- **No separate worker.** Background jobs are timers inside the API process (§15).
- **The API does not serve the frontend.** nginx (or a static host) serves `frontend/dist`. The browser calls the API on the same origin at `/api/v1`.

3. Repository map

```
server/
  index.js              entry point: env check, DB wait, migrations, tenant, boot seeds, routes, schedulers
  core/                 platform services shared by every module (§10)
  modules/
    performance/        the PMS itself, ~45 files (§11)
    people/             RnR, events/CSR, queries, career pathing, onboarding (§12)
    engagement/         surveys and insights (§12)
    agentic/            AI draft endpoints (§12, §13)
  migrations/           001-… 097-…, run in order at boot (§9)
  test/                 168 node:test files (§17)
frontend/
  src/App.jsx           the shell: sign-in, menu, routes, guards
  src/pages/            64 files, one per screen plus a few panels
  src/utils/api.jsx     the one fetch helper, phase labels, AI draft badge
  src/*.jsx             shared components (LongText, StatusTabs, SearchBox, KraTable…)
  test/                 21 browser tests (playwright-core), not run in CI
deploy/
  service/              systemd unit, nginx config, install.sh, update.sh, backups
  docker/               docker-compose alternative
  README.md             EC2 runbook
.github/workflows/deploy.yml   test → build screens → deploy to EC2 over AWS SSM
.claude/skills/         house rules as skills (conventions, migrations, permissions, shipping, extraction)
docs/                   Logic Book, manuals, deploy docs, change notes, this guide
render.yaml             Render blueprint (reference/dev only)
tools/                  sample-employees.csv / .xlsx for the importer
CONTEXT.md              running session log (history of decisions)
```

4. Running it locally

You need Node 22 and PostgreSQL 13 or later (CI uses 16).

```
# 1. A database
createdb apms_dev

# 2. The API
cd server
npm ci
DATABASE_URL=postgres://postgres:<pw>@127.0.0.1:5432/apms_dev \
DATABASE_SSL=false \
JWT_SECRET=<any-long-random-string> \
TENANT_SLUG=dev \
AUTH_DEV=true \
PORT=8080 \
node index.js
# On first boot it runs all migrations, creates the tenant "dev" and seeds its pages and permission bundles.

# 3. The screens (another terminal)
cd frontend
npm ci
npx vite --port 5190 --strictPort --host 127.0.0.1   # proxies /api to :8080 (vite.config.js)
```

- `AUTH_DEV=true` **is required.** It switches on the only sign-in route (`POST /auth/dev-login`), plus forgot/reset password. It is set on the live server as well. The name is historical: there is no other login method (§7).
- **Port.** Plain `npm run dev` uses Vite's default port 5173. The browser tests expect 5190, which is why the command above sets it.
- **First sign-in.** A fresh tenant has no employees, so the sign-in page offers **First-time setup** (`GET /setup/status`, `POST /setup/bootstrap-admin`, `server/core/setup.js`). It creates one Super Admin. After that:
  1. HR → Employees → import `tools/sample-employees.csv`. The import is a dry run until you confirm.
  2. Set logins: Employees → Manage → set a password, or the bulk credentials action.
  3. HR → Cycles → create a cycle and advance it through the phases.
- **No demo seed.** The repo has no script that loads demo data. The browser tests assume a demo tenant with users such as `emp@shot.in` and a password from `DEMO_PASSWORD`; you build that data yourself.
- **AI is optional.** Without `ANTHROPIC_API_KEY` the AI buttons return a clear "AI features are not set up" message and everything else works.
- **Email is simulated by default.** Mail mode starts as `simulated`: sends are logged in `core.notif_log` and nothing leaves the box (§14).

Every environment variable the server reads:

Variable	Required	Default	What it does
`DATABASE_URL`	yes	–	Postgres connection string
`JWT_SECRET`	yes	–	Signs session tokens; also derives the MoM recording key if `MOM_RECORDING_KEY` is unset
`TENANT_SLUG`	yes	–	The one tenant this server serves; created on first boot. Set once, never change.
`DATABASE_SSL`		on	SSL unless exactly `false`
`PORT` / `BIND_HOST`		`8080` / `0.0.0.0`	Listen address (EC2 uses `127.0.0.1` behind nginx)
`AUTH_DEV`		off	Must be `true` for sign-in, forgot and reset password
`TOKEN_TTL`		`12h`	JWT lifetime
`APP_URL`		–	Public URL used in email links
`ANTHROPIC_API_KEY`		–	Turns on AI drafts
`AI_MODEL`		`claude-opus-5-5`	Model for AI drafts (managed by `deploy/service/managed-settings.env`)
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`, `MAIL_PROVIDER`		–	Fallback mail settings when HR has not set them in the app
`APP_TIMEZONE`		`Asia/Kolkata`	Company time zone for dates and schedules
`MOM_STORAGE_DIR`, `MOM_RECORDING_KEY`		`/var/lib/agentic-pms/mom`	Meeting recording storage and encryption
`SERVER_PUBLIC_IP`		looked up	IP shown on HR's mail settings card
`GOOGLE_TOKEN_URL`, `GMAIL_API_BASE`		Google's	Overridden only in tests

The frontend reads one variable, `VITE_API_URL`. Leave it **unset** everywhere except the Render blueprint. It is baked in at build time, and the deployed screens call the API on their own origin.

5. How a request travels

`server/index.js` builds one Express app:

1. Global middleware: `cors()`, `express.json({ limit: '2mb' })`, and `req.tenantId` set to the boot-time tenant.
2. Routes, each with its own middleware chain:

Prefix	Chain (in order)
`GET /api/v1/health`	none: returns `{ok, service, build}`
`/api/v1/auth/*`, `GET /api/v1/brand`, `/api/v1/setup`	none (public)
`GET /api/v1/me`	`authenticate`
`/api/v1/employees`	`authenticate` → `scope-hooks.run()` → `apiPermissionParity` → `guardUuidParams`
`/api/v1/consent`, `/api/v1/gdpr`	`authenticate` → `apiPermissionParity` → `guardUuidParams`
`/api/v1/notifications`	`authenticate` → `guardUuidParams`
`/api/v1/pms`	`authenticate` → **HRBP gateway** → `apiPermissionParity` → `textLimits` → `guardUuidParams`
`/api/v1/people`, `/api/v1/engagement`	`authenticate` → `scope-hooks.run()` → `apiPermissionParity` → `textLimits` → `guardUuidParams`
`/api/v1/agentic`	`authenticate` → `scope-hooks.run()` → `apiPermissionParity`

What each step does:

- **`authenticate`** (`core/auth.js`) reads the JWT from `Authorization: Bearer` (or `?token=` for download links) and sets `req.user = {id, email, name, role, tenant_id, must_change_password}`. If the user must change their password, everything except `/me` and `/auth/password` returns 403.
- **HRBP gateway** (`modules/performance/hrbp-gateway.js`) lends an HRBP HR's permissions for one request, then narrows what they see and refuses writes outside their remit (§8). For the other routers it is registered through `core/scope-hooks.js`.
- **`apiPermissionParity`** (`core/permissions.js`) checks a route table, `core.route_permission`. **No migration seeds that table**, so on a normal install it allows everything and the handler checks below do the real work (§21).
- **`textLimits`** (`core/text-limits.js`) refuses over-long text on POST/PUT/PATCH with `422 {field, error}`. One-line fields allow 150 characters; other text allows 220 words and 2,000 characters. Meeting minutes, transcripts, templates and settings are exempt.
- **`guardUuidParams`** (`core/http.js`) returns 400 for a malformed `:id`-style parameter instead of letting Postgres throw.
- **Handler checks.** Each handler checks permissions itself (`hasPermission(req.user, 'pms_admin')` and so on) and applies row rules ("my reports only").

Errors. There is no global error handler. Each handler wraps its body in `try { … } catch (e) { res.status(500).json({ error: e.message }) }` and returns its own 4xx with a sentence the user can act on. 403s from the route table carry a `needs` field naming the missing permission.

6. Tenancy

- **One tenant per server.** `TENANT_SLUG` is looked up in `core.tenants` at boot and created if missing (`server/index.js`).
- **Every table has `tenant_id`, and every query filters on it,** even though a server has one tenant. This keeps a shared, multi-tenant deployment possible.
- **In handlers, the tenant comes from the token:** `const T = (req) => req.user.tenant_id;`. It is defined at the top of each module file. Public routes (sign-in, brand, setup) use `req.tenantId`.
- **Tenant settings** live in `core.admin_settings` (one JSON value per key: `smtp`, `mail_send_mode`, `brand`, `timesheet`, `mom`, `super50_window`…). HR edits the whitelisted ones through `GET/PUT /api/v1/pms/hr/settings`.

7. Sign-in and sessions

Server (`core/auth.js` , `core/password-reset.js` , `core/setup.js`):

- **Email and password only**, checked with bcrypt against `core.local_credentials` . There is no SSO.
- `POST /auth/dev-login` returns `{token, user}` and works only with `AUTH_DEV=true` .
  - The token is a JWT (HS256, `JWT_SECRET` , 12 hours by default) carrying `sub, email, name, role, tenant_id, pwc` .
  - **The role is inside the token**, so a role change applies at the next sign-in.
  - Only employees with status `active` can sign in.
- **Passwords:**
  - At least 8 characters.
  - A password set by HR forces a change at first sign-in (`must_change_password`).
  - Forgot password emails an 8-character code that is valid for 30 minutes. An address can request 3 codes per hour; each code allows 5 attempts.
- **First admin:** `POST /setup/bootstrap-admin` works only while the tenant has no employees.

Browser (`frontend/src/App.jsx` , `utils/api.jsx` , `IdleGuard.jsx`):

- **Token storage.** The token is in `localStorage.apms_token` . On load the app calls `GET /me` , which returns the user and `pages` : the routes this person may open, from `core.page_permission` . If `/me` fails, the token is dropped.
- **Idle sign-out** after 30 minutes, with a warning 2 minutes before. It is shared across tabs.
- **Forced password change.** When `must_change_password` is set, the change-password screen replaces the app.
- **New-build notice.** `BuildWatch` polls `/api/v1/health` every 5 minutes. When the server's build differs from the one the screens were built from, it shows a *Reload now* bar.

8. Permissions and whose rows you see

There are two separate questions: **may I open this screen or call this route**, and **which people's rows do I see**.

Permissions

- **Effective permissions** = `core.role_permissions[role]` ∪ `core.user_permissions[email]`; `*` means everything (`core/permissions.js`).
- **Roles** are stored in `core.user_roles` (keyed by email); no row means `employee`.
- **Default bundles** are seeded by `migrations/002` and re-applied at every boot with `ON CONFLICT DO NOTHING`. A missing grant comes back; an added one is kept. A client changes access by editing rows, never code.

Role	Permissions
employee	`pms_self`, `engagement_take`, `people_view`
manager	employee + `pms_team_eval`
hod	manager + `pms_hod`
hrbp	`pms_self`, `pms_hrbp`, `people_view`, `engagement_take`, `onboarding_ops`, `pms_compensation`
hr	`pms_admin`, `pms_team_eval`, `pms_hod`, `pms_hrbp`, `engagement_admin`, `people_admin`, `letters_admin`, `pms_compensation`, `onboarding_ops`, …
hr_ops	`pms_self`, `engagement_take`, `people_view`, `onboarding_ops`
admin	`*` (shown as "Super Admin")

Functions:

- **`hasPermission(user, perm)`** is what handlers call.
- **`holdsPermission(user, perm)`** ignores permissions the HRBP gateway lends for one request. Use it where an HRBP must never act: roles, passwords, the employee import, branding, remits.

Screens

- **`core.page_permission`** has one row per screen: `(page, route, required_permission)`, where `NULL` means everyone.
- That one row drives the menu and the direct-URL guard alike, so they cannot disagree. `/me` returns the allowed routes, the menu shows only those, and opening any other route shows "This page is not part of your access".
- New pages are added by a migration's `ensure…Pages` function, called at boot (§19).

Rows

Who	Sees
Employee	Their own records
Manager	Direct reports (`core.employees.manager_id`)
HOD	Their departments (`core.department_heads`)
HRBP	Their remit (`core.hrbp_scope`: by location or by HOD)
HR / Super Admin	Everyone in the tenant

The HRBP gateway (`modules/performance/hrbp-gateway.js`) is how the HRBP pages reuse HR's handlers without editing each one:

- It lends `pms_admin`, `people_admin` and `engagement_admin` for one request.
- It filters results to the remit.
- It refuses company-wide settings writes, and refuses any write for a person outside the remit.
- HRBP screens have their own routes (`/hrbp/*`) so the HR section never appears in an HRBP's menu.

Logic Book §1.2–1.3 has the full rules.

9. Database and migrations

Schemas

Schema	Holds
`core`	tenants, employees (the mirror), department heads, departments, roles and permissions, page permissions, credentials, notifications, mail log, settings, audit log, consents, HRBP remits, password resets, `migrations_log`
`pms`	cycles, KRA sheets and KRAs, self appraisals, mid-year, manager and HOD evaluations, rating adjustments, performance history, competencies, development and career plans, connects, minutes, PIPs, timesheets, 9-box, Super 50, increments, closure letters, KRA library, audit log
`people`	events, CSR, queries, career paths and transitions, campus, onboarding, legacy awards
`rnr`	awards, cycles, allocations, nominations, events, quota overrides
`engagement`	surveys, questions, invitations, responses, answers, templates, flag rules
`agentic`	`drafts` (every AI output and its input), `recommendations`

The employee table is a mirror. `core.employees` is loaded from the client's HR system export through the importer (§10). It is not an HRMS: there is an edit form for single-field fixes, but email is never edited (credentials and roles are keyed by email).

The migration runner

`server/core/migrate.js` runs at every boot, before the server listens:

- **Files.** It loads `server/migrations/NNN-name.js` in name order. There are 97, `001-people-core.js` to `097-mom-ai-edit.js`. Each file exports `async up(db)`.
- **Tracking.** Files already listed in `core.migrations_log` are skipped.
- **Failure.** Any error stops the boot. A schema the code expects but doesn't have is a broken deploy, so it refuses to start rather than run half-migrated.
- **No transaction.** It doesn't wrap a migration in one, so every statement must be safe to run twice (`IF NOT EXISTS` everywhere).
- **Standalone.** `npm run migrate` in `server/` runs it without starting the server.

Rules (from `.claude/skills/apms-migrations`):

- Never edit or renumber a shipped migration; add a new one.
- `CREATE TABLE IF NOT EXISTS` skips an existing table, so add each new column with its own `ADD COLUMN IF NOT EXISTS`.
- `ON CONFLICT` needs a real unique constraint: create the index in the same migration.
- Backfills touch only unambiguous rows and NULL targets, and log how many they changed.

Boot seeds

Migrations that loop over `core.tenants` find nothing on a fresh install, because the tenant row is created *after* migrations run. So `server/index.js` calls each seed function directly for the boot tenant on every boot:

- `002.ensureTenantSeeds`: permission bundles
- `042.ensurePageSeeds`: pages
- `077.seedFor`: RnR masters
- 14 more, from `008` (default parameters) to `096` (query pages)

All are idempotent. If your migration seeds per-tenant data, export the seed function and call it here too.

10. Server: core services

`server/core/` is shared by every module. Modules import core; core is not supposed to import modules (§21 lists where it does).

File	What it is
`db.js`	`pg` pool (10 connections), `query`, `getClient` for transactions, `waitForDatabase` (15 tries × 4 s at boot)
`migrate.js`	The migration runner (§9)
`auth.js`	JWT sign/verify, `authenticate`, sign-in, change password
`password-reset.js`	Forgot / reset password with an emailed code
`setup.js`	One-time first admin for an empty tenant
`permissions.js`	`effectivePermissions`, `hasPermission`, `holdsPermission`, the route-table gate
`scope-hooks.js`	Lets a module (the HRBP gateway) narrow core and other routers without core importing it
`http.js`	`guardUuidParams`
`text-limits.js`	Server-side text limits (mirrors `frontend/src/LongText.jsx`)
`employees.js`	The employee mirror: import (CSV/XLSX, dry run by default), edit, archive/restore/erase, logins, roles, departments and heads
`notifications.js`	`notify(tenantId, employeeId, kind, title, body, link, {email})` and the bell's routes
`mail.js`	`sendMail` with simulated / live mode, SMTP or Gmail transport, every send logged
`gmail.js`	Gmail API send-as through a service account
`ai.js`	`narrate()`: the one way to call the AI model (§13)
`brand.js`	Company name and logo (`GET /brand` is public)
`consent.js`	Recorded consents (e.g. AI insights on meetings)
`gdpr.js`	A person's data export (JSON or a readable XLSX)
`meetings.js`	Meeting-provider seam (manual only; Google Meet declared, not available)
`bulk-credentials.js`	Password generation for bulk logins
`logger.js`	One JSON line per log entry to stdout/stderr

The employee import (`POST /api/v1/employees/import`, people_admin, held):

1. **Dry run first.** It runs as a dry run unless `?commit=1`. The preview lists per-row problems, role grants and KRA auto-assignment.
2. **Validation** (`validateEmployeeRows`) checks:
   - required name and email, with common header names recognised;
   - duplicate emails;
   - managers by email or name, and manager-chain cycles;
   - HOD consistency within a department.
3. **Commit** is one transaction. It upserts on email, links managers, hands open records to a new manager, reopens KRA sheets when the job changed, and grants manager or HOD roles.

Audit. Every change that can affect a rating is written to an audit table, so "why did my rating change?" always has an answer.

- There is no shared audit helper. Each module defines a local `audit()`.
- `performance` writes `pms.audit_log` with `(action, cycle_id, employee_id, details)`.
- `people` and `engagement` write `core.audit_log`.
- RnR writes `rnr.events`.

11. Server: the performance module

`server/modules/performance/` is the PMS itself: 43 files, about 21,000 lines, 263 routes, all under `/api/v1/pms`.

- **`index.js`** holds about 150 of those routes, and the rest live in sub-routers it mounts.
- **The `*-rules.js` files and `phase-machine.js` are pure:** no database, unit-tested directly. Put new calculations there, not in a handler.

Files

Group	Files
Router and the cycle	`index.js` (cycles, KRAs, KRA library, HR settings and mail, growth plan, self appraisal, mid-year, evaluations, HOD, calibration, publish, compensation, connects, meetings, PIP, watchlist, letters, reports), `active-cycle.js` (which cycle is "the" cycle), `phase-machine.js` (phase rules)
Ratings and rules (pure)	`rating-rules.js` (weighted rating, Super 50 rule, default scale), `grade.js` (number → letter for PDFs and exports), `eligibility.js` (who is in this cycle by joining date), `review-form.js` (mid-year form questions)
Approvals and home	`approvals.js` (approve or return KRA sheets and growth plans; HR's All Approvals queue), `home.js` (the Dashboard's "what you owe")
KRA support	`kra-autoassign.js` (library KRAs onto a new hire's sheet), `kra-goal-sync.js` (re-link growth goals when KRAs change), `kra-keywords.js`, `profile-change.js` (reopen a locked sheet when the job changes), `manager-handover.js` (move open records to a new manager), `prior-ratings-import.js`
Scope	`team-scope.js`, `reporting-tree.js` (everyone below a person), `hrbp-gateway.js`, `hrbp.js` (HRBP tab)
Connects and meetings	`connect-reminders.js`, `connect-ai.js`, `mom.js` (minutes of meeting tab and its scheduler), `mom-store.js` (encrypted recordings), `mom-transcript.js`
Competencies	`competencies.js` (routes), `competency-rules.js` (pure)
Calibration, pay, talent	`calibration-budget.js` (the "kitty", in paise), `increment-rules.js`, `nine-box.js` + `nine-box-rules.js`, `super50.js`, `super50-benefits.js`, `hod-team.js`
Reminders	`reminders.js` (engine), `reminder-schedule.js` (April–March calendar, pure)
Timesheet	`timesheet.js` (upload, compliance), `timesheet-rules.js`, `timesheet-template.js`, `timesheet-kra.js` (hours → KRAs, month close, override), `timesheet-kra-match.js`, `timesheet-kra-score.js`, `timesheet-rollup.js`

Sub-routers mounted from `index.js`:

Prefix	File
`/competencies`	`competencies.js`
`/timesheet`	`timesheet.js`
`/timesheet/kra`	`timesheet-kra.js`
`/hrbp`	`hrbp.js`
`/nine-box`	`nine-box.js`
`/team/evaluations/:id/potential`	`nine-box.js`
`/mom`	`mom.js`
`/hod/team`	`hod-team.js`
`/super50`	`super50-benefits.js`

The cycle and its phases

`phase-machine.js`:

```
draft → kra_open → mid_year_review → self_appraisal → manager_eval → hod_eval → calibration → publish → closed
```

- **Moving between phases.** HR moves the cycle with `POST /cycles/:id/phase {to | rollback | cancel}` (`pms_admin`), and it is audited.
  - Forward and back go one step at a time. `cancelled` is reachable from any phase but `closed`.
  - Each phase change notifies the people it opens for.
- **`phaseAllows(phase, action)`** decides what a phase opens:
  - **KRA editing** is open in every phase except `draft` and `closed`; the sheet's own status decides who writes.
  - **Mid-year** opens at `mid_year_review` and stays open through `self_appraisal`.
  - **Single-phase actions:** self appraisal (`self_appraisal`), manager evaluation (`manager_eval`), HOD rating (`hod_eval`), calibration adjustments (`calibration`) and publish (`publish`) each open only in their own phase.
- **Other gates.** Growth plans, career aspirations and editing KRAs during a review have their own rules (`growthEditable`, `aspirationEditable`, `reviewKraEditable`).
- **Cycle types.** `cycle_type` is `annual` or `midyear`. 9-box, Super 50 and the potential step run only on annual cycles. A mid-year publish opens a PIP below `pip_threshold`.
- **The active cycle.** `activeCycle()` (`active-cycle.js`) returns the newest cycle that isn't closed or cancelled, preferring started cycles over drafts. Use it; don't write your own query.

One employee through the cycle

Step	Who	Route (under `/api/v1/pms`)	Table and statuses
KRA sheet	Employee	`GET /my/kra-sheet` (creates it), `PUT /my/kra-sheet/kras`, `POST /my/kra-sheet/submit` (weights must total 100)	`pms.kra_sheets` `draft → submitted → approved` or `returned`; `pms.kras`
Approve KRAs	Manager (live `manager_id`) or HR	`POST /team/kra-sheets/:sheetId/decide`; manager edits `PUT /team/kra-sheets/:sheetId/kras` (audited diff)	same

Step	Who	Route (under `/api/v1/pms/`)	Table and statuses
Growth plan	Employee → manager	`/my/development-plan…`, `POST /team/development-plans/:planId/decide`	`pms.development_plans`, `pms.development_goals`
Mid-year	Employee, then manager	`/my/midyear-review…`, `/team/midyear-review/:employeeId…`	`pms.midyear_checkins` (per-KRA entries keyed by `kra_id`)
Self appraisal	Employee	`/my/self-appraisal`, `…/submit`, `…/evidence`	`pms.self_appraisals` `not_started → in_progress → submitted`; `pms.evidence` (files)
Manager evaluation	Manager	`PUT /team/evaluations/:employeeId`, `…/submit`; HR `…/reopen` with a reason	`pms.manager_evaluations` `pending → submitted` (locked)
Potential	Manager (annual)	`PUT /team/evaluations/:employeeId/potential`	`pms.potential_answers`
HOD review	Department head	`GET /hod/queue`, `PUT /hod/queue/:employeeId` (`{overall_rating, comment, submit}`)	`pms.hod_evaluations` `pending → submitted` (locked)
Calibration	HR	`GET /calibration`, `POST /calibration/adjust` (reason required)	`pms.rating_adjustments` (latest row counts)
Publish	HR	`POST /publish`	`pms.employee_performance_history`, plus the steps below

Competencies (`/competencies/me`, `/competencies/team/:employeeId`) run alongside the cycle and are not phase-gated. Connects, PIPs and minutes of meeting are not tied to a phase.

Ratings

- **Where the scale lives.** Each cycle has its own `rating_scale` (JSON). New cycles get `5 A+, 4 A, 3 B+, 2 B, 1 C`, a bell curve, and a PIP threshold of 3.0.
- **Stored as numbers, shown as letters.** The database stores numbers. The screens show letters (`frontend/src/grade.jsx`); the server does the same for PDFs and exports (`grade.js`).
- **Weighted ratings.** The self and manager overalls are weighted averages of per-KRA ratings: Σ rating × weight / 100, rounded to 0.1 (`computeWeightedRating`).
- **The final rating** is `COALESCE(latest HR adjustment, submitted HOD rating, submitted manager rating)`. The same expression appears in calibration, publish, the 9-box and the employee's rating chain; keep them in step if you ever change it.
- **Withheld until publish.** An employee sees no manager, HOD or HR rating until a history row exists for them (`publishedFor`).

What publish does

`POST /publish` runs in the `publish` phase, `pms_admin`, and is safe to repeat:

1. Writes `pms.employee_performance_history` (final rating and label) for everyone with a submitted manager evaluation, and updates `core.employees.last_appraisal_*`.
2. Creates a closure-letter row for each person.
3. **Annual cycles:**
   - recomputes the Super 50 flag and alerts HR about retention risks;
   - places everyone on the 9-box, with actions, succession pool and automatic PIPs (`nine-box.js publishPlacements`);
   - assigns Super 50 benefit tiers.
4. **Mid-year cycles:** opens a PIP for anyone below `pip_threshold`.
5. Notifies everyone published and writes `CYCLE_PUBLISHED` to the audit log.

Timesheet → KRA

1. **Upload.** Employees, managers or HR upload a Zoho Sprints export at `POST /timesheet/upload`.
   - It is a dry run unless `?commit=1`.
   - A re-upload replaces only the dates present in the file.
   - Non-HR uploaders load only their own rows.
2. **Compliance** is hours logged ÷ (8 h × working days) per period. The period starts on day 21 by default. It is graded Green / Amber / Red against settings.
3. **Matching to KRAs.** Work items are matched to KRAs in this order: explicit mapping, then KRA title, then keyword.
   - The manager or HR maps them (`/timesheet/kra/employee/:id/map`).
   - HR closes months (`POST /timesheet/kra/close`).
   - A manager or HR can override a closed month with a reason.
4. **Calibration** shows the timesheet rollup as context only. It never feeds the rating.

Details and formulas: Logic Book §3.7.

Who may do what

There is no single guard. Look for these helpers before writing your own:

Helper	
`hasPermission` / `holdsPermission`	
`teamScope`, `mayRead` (`team-scope.js`)	
`reportingTree` (`reporting-tree.js`)	
`hodMayRate`, `hodDepartments`	

Helper	
`reviewKraAccess`, `isManagerOfOrAdmin`, `maySignOff`, `meetingParty`	
`submittedByMe`	
`canRead` / `canWrite` (`timesheet-kra.js`), `assessable` (`competencies.js`), `scopeFor` (`nine-box.js`), `roleIn` / `editable` (`mom.js`), `hrOnly` (`super50-benefits.js`)	

Two rules to keep:

- **Manager checks use the live manager** (`core.employees.manager_id`), not the copy stored on the record.
- **Nobody sees their own 9-box placement**, HR included.

Importers

The KRA bulk upload, KRA library, compensation sheet, prior ratings and timesheet uploads all work the same way as the employee import:

- a dry run unless `?commit=1`;
- a per-row report of what would load and what would not.

Keep that shape for any new importer.

12. Server: people, engagement and agentic

people (`/api/v1/people` , `server/modules/people/`)

Feature	Files	Notes
Rewards & Recognition (RnR)	`rnr.js` ; pure engines `rnr-eligibility.js` , `rnr-quota.js` , `rnr-workflow.js`	See below
Events and CSR	`index.js`	HR creates (`people_admin`); everyone RSVPs or signs up on People Hub
Queries	`queries.js`	Raised to colleagues or HR; open → answered → closed; email to recipients
Career pathing	`index.js` , `career-*.js`	HR's transition matrix (bulk upload with dry run); employee's short- and long-term path and milestones, edits gated by cycle phase
Onboarding (First-Week Journey)	`onboarding*.js`	Joiners, day-by-day tasks, SPOC emails. Hidden from the menu since 8 Oct by `SHOW_FIRST_WEEK_JOURNEY` ; data kept
HRBP remit	`hrbp-scope.js`	Location OR HOD; fails closed. Exported as `hrbpScope`
Campus drives, legacy awards	`index.js`	Older features; People Hub's awards now come from RnR

RnR in one paragraph.

- **Setup.** HR sets up award masters and opens a cycle (quarterly, half-yearly or annual). Opening freezes the headcount and the quota: 3% by default.
- **Nominating.** A manager nominates one of their reports. Eligibility is worked out from the employee master (tenure, experience, previous awards) and a person who doesn't qualify is shown with the reason.
- **Approval chain.** Defined in `rnr-workflow.js` : **HOD → HRBP → HR**. Statuses are `pending_delivery_head` , then `pending_hrbp` , then `pending_hr` , then `final_approved` , then `awarded` ; `sent_back` and `rejected` need a reason.
- **Approver rules.** Nobody approves a nomination they raised. An HOD acts only for their departments. An HRBP acts only within their remit.
- **Quota.** HR's final approval is checked against the quota. Going over needs an override reason, which is recorded in `rnr.quota_overrides` .
- **History.** Every action is written to `rnr.events` .

engagement (`/api/v1/engagement` , `server/modules/engagement/`)

- **Surveys** go draft → open → closed. They are built from a template library (`templates.js`): pulse, Day 1 to Day 90 lifecycle surveys, manager-about-reportee, exit, stay interview, eNPS…
- **Audience.** A survey's audience is either a rule (departments, designations, bands, managers, tenure window; `audience.js`) or tenure-triggered. Tenure-triggered surveys are swept daily (§15).
- **Anonymity is structural.**
  - `engagement.invitations` records who was asked and whether they finished.
  - `engagement.responses` stores `employee_id = NULL` unless the survey is attributed, or the person opted in.
  - Completion is written on the invitation, never the response.
  - Exports withhold any group smaller than 5.
- **Insights** (`insights.js`) are plain arithmetic over attributed answers: flags, dimension scores, new-hire index. They involve no AI.

agentic (`/api/v1/agentic` , `server/modules/agentic/index.js`)

About 20 endpoints that return **draft text**. Examples:

- appraisal and mid-year drafts
- connect insights and auto-tags
- KRA, development-plan and career suggestions
- justification review, review assist, meeting summary, appraisal summary
- calibration brief, cycle health, engagement themes, letter drafts

Each checks the same access as the screen it serves (for example, a manager only for their own reports). How they work is in §13.

Module boundaries

The rule is that modules import core, and use each other only through what a module exports from its `index.js` . Exported interfaces:

- **people** exports `hrbpScope` and the career readers (`careerPathFor` , `targetsFor` , `experienceFrom` …).
- **performance** exports `buildAnnualReviewSummary` and `kraSuggestionCandidates` .
- **engagement** exports `sweepTenureSurveys` .

`performance/active-cycle.js` is the one place that decides which cycle is active, and other modules use it. §21 lists where the boundary rule is broken today.

13. AI features

All AI calls go through `core/ai.js` → `narrate()`:

- **The call.** It posts to the Anthropic Messages API with `fetch`; there is no SDK dependency. The model is `AI_MODEL` (default `claude-opus-5-5`) and the key is `ANTHROPIC_API_KEY`.
- **The record.** It stores every call in `agentic.drafts`: kind, reference, the **input verbatim**, the output, the model, and who asked. So any draft can be traced to what produced it.
- **Labels.** It marks every result `_draft: true`. The screens show "AI DRAFT — edit before use" (`DraftBadge` in `utils/api.jsx`).
- **No ratings.** It strips any rating-like key from the model's answer (`stripRatingSuggestions`). Ratings, scores and distributions are always SQL; AI only writes text.
- **Length.** Drafts that land in a review field are trimmed to 220 words (`fitWords`) to match the text limits.
- **Failures.**
  - With no key, the call fails with 503 `not_configured` and a sentence the user can act on.
  - Errors from the AI service map to 502 with a safe message and `reason` (`auth`, `rate_limited`, `account`, `upstream`, `bad_request`).
- **Boot check.** The server logs whether the key is set and its fingerprint, never the key.

AI features hosted inside `performance` also use `narrate()`:

- connect categorisation and training suggestions (`connect-ai.js`);
- the MoM draft from a transcript (`mom.js`).

Adding one is a recipe in §19.

14. Notifications, mail and files

- **In-app notifications.** `notify(tenantId, employeeId, kind, title, body, link, { email })` (`core/notifications.js`) writes a row in `core.notifications` for the bell.
  - With `{ email: true }` it also emails, skipping inactive people and placeholder addresses.
  - A mail failure is logged and never breaks the action.
  - The bell (`frontend/src/pages/NotificationBell.jsx`) polls every 60 s.
- **Mail.** `sendMail` (`core/mail.js`) has two modes, stored in `core.admin_settings.mail_send_mode`:
  - **simulated** (the default): logs and sends nothing;
  - **live**: sends through SMTP (nodemailer) or the Gmail API (send-as through a service account).
  - Every attempt is logged in `core.notif_log` with outcome `sent`, `simulated` or `failed`.
  - HR configures and tests it in HR → Settings → Email.
- **Files.**
  - Uploads are held in memory (multer), with size limits per route: 5 MB for the employee import, 10 MB for evidence.
  - Evidence and closure letters are stored as `bytea` in Postgres.
  - Meeting recordings go to disk under `MOM_STORAGE_DIR`, encrypted with AES-256-GCM (`modules/performance/mom-store.js`).
- **Downloads** are plain links with `?token=` because a browser link cannot send a header; `authenticate` accepts both.
- **Spreadsheets.** Excel is generated with ExcelJS and PDFs with PDFKit. SheetJS (`xlsx`) is deliberately not used.

15. Background jobs

The API process starts these in `server/index.js`. Each runs once at boot, then on a timer.

Job	Every	Where
Connect reminders	24 h	`modules/performance` (`checkAndSendConnectReminders`)
Calendar reminder sweep (ledger in migration 026)	24 h	`modules/performance` (`runReminders`)
Tenure-triggered survey sweep	24 h	`modules/engagement` (`sweepTenureSurveys`)
MoM scheduler	5 min	`modules/performance/mom.js`
Super 50 benefits	6 h	`modules/performance/super50-benefits.js`

There is one API process per server. If you ever run two, these jobs run twice; the reminder ledgers and `ON CONFLICT` inserts are what keep repeats harmless.

16. Frontend

The shell (`frontend/src/App.jsx`)

- **Menu.** `NAV` is the menu: groups (Self, Manager, HOD, HRBP, HR, HR Ops) of `{to, label, icon}` items, with Dashboard (`/home`) above them.
- **Which items show.** An item shows only if its route is in `user.pages` from `/me` . Empty groups disappear.
  - Some items also have a **gate**. For example, *My Rating* is greyed until a rating is published, and *My Super 50* is hidden unless the person is in it.
  - Feature switches that hide whole screens are in `src/features.js` .
- **Routes.** There is one flat `<Routes>` table. Pages take `user` as a prop only where they need it.
- **URL guard.** The URL guard finds the menu item whose route best matches the address. If the person may not open it, `NoAccess` renders instead of the page.
  - `RequireRole` adds a client-side role check on five HR pages.
  - The server enforces access regardless.
- **Top bar.** Page search, the notification bell, help (shows the build), the account menu.

Talking to the API (`frontend/src/utils/api.jsx`)

- **`api(path, opts)`** .
  - Adds `Authorization: Bearer <apms_token>` , sends JSON, and parses the JSON reply.
  - On a non-2xx reply it throws `Error(data.error)` with `.status` and `.data` ; show `e.message` to the user.
  - There is no global 401 handler.
- **Downloads** are `<a href={` /api/v1/...?token=${token} `}>` .
- **Shared exports.** The same file exports `PHASES` , `phaseLabel` , `phaseColor` , `sheetStatusLabel` and `DraftBadge` .

Shared components (`frontend/src/`)

Component	Use it for
`LongText.jsx`	Any multi-line input: 220-word limit with a counter; also exports `Req` (the red required-field asterisk)
`StatusTabs.jsx`	Status filter chips with counts; every tab always shows
`SearchBox.jsx`	The one list search; `matches(q, ...fields)` filters in the browser
`PageHead.jsx`	The page title band
`Kpi.jsx`	Dashboard count cards
`KraTable.jsx`	The one KRA table (parameters, KRAs, KPIs, weightage)
`grade.jsx`	Shows ratings as letters (A+ ... C); the number goes in the tooltip. A test enforces letters on screen.
`PasswordInput.jsx`	Password field with show/hide; paste is blocked
`TimesheetDashboard.jsx` , `TimesheetKra.jsx` , `TimesheetRating.jsx` , `TimesheetTabs.jsx`	Timesheet views shared by Self, Manager and HR
`IdleGuard.jsx` , `Brand.jsx` , `Summit.jsx` , `CompetencyScale.jsx` , `EmailSettings.jsx` , `KraRatingSettings.jsx`	As named

Styling

- **Tailwind** with custom colours `navy` , `brand` , `lagoon` , `amber2` and `leaf` (`tailwind.config.js`).
- **Design classes** in `src/app.css` : `card` , `btn-pri` , `btn-sec` , `inp` , `lbl` , `chip` , `pill-*` , `tbl` , `kpi` .
- **Missing shades.** A colour shade that doesn't exist (e.g. `leaf-700`) compiles to nothing, silently. Check the config.
- **Empty states** are written inline as a centred `card` , with a sentence saying why the list is empty and what to do.
- **Numbers come from the server.** The browser never recomputes a score.

17. Tests

Server (`server/test/` , 168 files, `node:test`)

```
cd server
DATABASE_URL=postgres://postgres:<pw>@127.0.0.1:5432/apms_test DATABASE_SSL=false npm test
# single file:
env -u JWT_SECRET -u TENANT_SLUG -u AUTH_DEV \
  DATABASE_URL=postgres://postgres:<pw>@127.0.0.1:5432/apms_test DATABASE_SSL=false \
  node --test test/text-limits.test.js
```

- **Serial.** `npm test` runs the files one at a time (`--test-concurrency=1`) because they share one database.
- **Use a throwaway database.** The DB tests run the migrations themselves, create their own tenant and sign in through `dev-login` . Unset `JWT_SECRET` , `TENANT_SLUG` and `AUTH_DEV` in your shell, as above, so the tests' own values apply.
- **Unit tests without a database.** About 40 files are pure unit tests: phase machine, eligibility, quota, text limits, importer validation, and so on. They need no database.
- **Missing skip guard.** Most DB tests skip when `DATABASE_URL` is unset, but ten files don't check it and fail instead (§21).

Frontend (`frontend/test/` , 21 files)

- **What they are.** Browser tests with `playwright-core` against a running API (`:8080`) and Vite (`:5190`) with demo data.
- **When they skip.** They skip if the stack is not up.
- **Running them.** Run with `cd frontend && npm test` . `CHROME_PATH` , `APP_URL` , `API_URL` and `DEMO_PASSWORD` override the defaults.
- **Not in CI.** CI does not run them; run them yourself before shipping screen changes.
- **One static test.** `ratings-are-letters.test.mjs` scans the source and needs no stack.

Before you push

1. `node --check` on the server files you touched.
2. The affected server tests, then the full `npm test` .
3. `npm run build` in `frontend/` .
4. The secrets scan in `.claude/skills/apms-secrets-and-shipping` .

18. CI and deployment

`.github/workflows/deploy.yml` runs on every push to `main`, or by hand for a branch:

1. **resolve**: pins one commit SHA for every later job.
2. **test**: the full server suite against a `postgres:16` service. A red test stops the deploy.
3. **web-build**: builds the screens with `VITE_API_URL` unset and stamps them with the commit.
4. **web-publish**: stores the built screens as a git tag `web-build/<sha>`, keeping the last 30, so the server never builds the frontend itself.
5. **preflight**: skips the deploy (with a notice) if the AWS secrets aren't set.
6. **deploy**:
   - Signs in to AWS with OIDC; no stored keys.
   - Runs `deploy/service/update.sh <branch> <sha>` on the EC2 box through **AWS SSM**; there is no SSH.
   - Checks that `/api/v1/health` answers 200.

Secret names only: `AWS_DEPLOY_ROLE_ARN` and `EC2_INSTANCE_ID`; variable `AWS_REGION`.

`deploy/service/update.sh` on the box:

1. Takes a lock.
2. Backs up the database.
3. Fetches and refuses to go backwards (an older commit prints "Already past").
4. Downloads that commit's screens.
5. Fast-forwards `/opt/agentic-pms`.
6. Runs `npm ci` if the lockfile changed.
7. Syncs managed settings into `/etc/agentic-pms/api.env`.
8. Copies the screens to `/var/www/agentic-pms`.
9. Restarts `agentic-pms-api`, which runs the migrations.
10. Waits up to 90 s for health and checks that nginx serves the new bundle.

It ends with either "Healthy. Deployed A → B" or "NOTHING NEW was deployed".

Which build is live?

- `GET /api/v1/health` returns `build`, the short SHA of the checkout the API is running.
- The screens carry the same SHA, shown in the **?** menu.

Merging to `main` is deploying: the workflow deploys every push.

Other ways to host.

- `deploy/docker/docker-compose.yml`: db, api and web.
- `render.yaml`: a Render blueprint for reference; free-tier Postgres is not durable.
- A new client install: `deploy/service/install.sh`, `docs/NEW-CLIENT-DEPLOYMENT.md`.

19. Recipes: making common changes

Add an API endpoint

1. **Pick the module** and put the route in the file that owns the feature. A new area gets its own file and a sub-router mounted from the module's `index.js`.
2. **Write the handler in the house shape:**

```
router.post('/things', async (req, res) => {
  try {
    if (!(await hasPermission(req.user, 'pms_team_eval'))) return res.status(403).json({ error: "Requires 'pms_team_eval'", needs: 'pms_team_eval' });
    const { name } = req.body || {};
    if (!name) return res.status(422).json({ error: 'Give the thing a name.' });   // a sentence the user can act on
    const row = (await db.query(
      `INSERT INTO pms.things (tenant_id, name, created_by) VALUES ($1,$2,$3) RETURNING *`,
      [T(req), name, req.user.id])).rows[0];
    audit(req, 'THING_CREATED', null, null, { id: row.id });          // if it can affect a rating
    res.json({ thing: row });
  } catch (e) { res.status(500).json({ error: e.message }); }
});
```

3. **Scope rows in the handler** (my reports, my department, remit), not in the route table.
4. **Text fields.**
   - If a new one-line field is free text, add its key to `SHORT_KEYS` in `core/text-limits.js`.
   - If a field is long by nature, add it to `SKIP_KEY`, or its path to `SKIP_PATH`.
5. **HRBP use.** If HRBPs will use the route through `/hrbp/*` screens, read `hrbp-gateway.js`:
   - a configuration route belongs in `TENANT_WIDE`;
   - a write must resolve to a person in the remit.
6. **Add a test** in `server/test/` with the `HAS_DB` skip guard.

Add a screen

1. **The page.** Create `frontend/src/pages/ThingPage.jsx` using `PageHead`, `api()`, and `LongText`, `SearchBox` and `StatusTabs` where they fit.
2. **The route and menu item.** Add a `<Route>` in `App.jsx` and an item in the right `NAV` group.
3. **The permission row.** Register the page with a new migration that exports an `ensure…Pages(db, tenantId)` function. Copy `migrations/096-queries.js`. Call it from `up()` for existing tenants **and** from `server/index.js` for the boot tenant. Without the row, the menu won't show the page; with a wrong permission, the URL guard blocks it.

Add a migration

1. **The file.** Create `server/migrations/098-what-it-does.js` exporting `async up(db)`. Make every statement idempotent (§9).
2. **Per-tenant data.** If it seeds per-tenant data, export the seed function and call it at boot in `server/index.js`.
3. **Run it twice.** Boot the server twice against the same database; the second boot must change nothing.

Add a setting HR can change

1. Add the key to the `SETTINGS` whitelist in `modules/performance/index.js` (`PUT /hr/settings/:key`).
2. Read it from `core.admin_settings` with a sensible default when the row is absent.
3. Thresholds, labels and lists are settings or table rows, never constants in code.

Send a notification

```
await notify(T(req), employeeId, 'thing_ready', 'Title in plain words', 'One-line body', '/route/to/open', { email: true });
```

Add the link mapping in `NotificationBell.jsx` if the link is an old route.

Add an AI draft

Call `narrate()` from `core/ai.js` with a `kind`, a `ref`, the input object and a prompt. Then:

- Return the draft with the AI DRAFT label.
- Never write the model's output into a rating field.
- Let the 503 / 502 errors pass through to the screen.

20. House rules

These come from `.claude/skills/` and the Logic Book. Code review holds you to them.

1. **`tenant_id` on every table and in every WHERE.**
2. **No silent failure.** Every `catch` fixes the problem or surfaces it. Batch operations return per-row reasons; the employee importer is the reference. Never `.catch(() => {})` inside a transaction.
3. **Deterministic numbers; AI narrates.** Ratings, scores and distributions are SQL. AI writes labelled drafts, stored with their input.
4. **Thresholds, labels and dropdowns live in tables.** Clients configure; they never fork.
5. **Every change that can affect a rating is audited.** "Why did my rating change?" must have a queryable answer.
6. **Errors are short, plain sentences for users; the full detail goes to the logs.** 403s from the route table keep the `needs` field.
7. **Anonymity in engagement is structural.** Invitations and responses stay apart; nothing joins them for an anonymous survey.
8. **Modules import core, and use each other only through exported interfaces.**
9. **Secrets never enter the repo.** Not in code, docs or commit messages; document the variable name only.
10. **One concern per commit.** The message says what, why, the root cause for a fix, and how it was tested. Client engineers will read the history.
11. **Ratings show as letters** (`grade.jsx`); the stored number is never shown on its own.

21. Known gaps and gotchas

These were found while writing this guide (20 Oct 2026). Each is checked against the code. They are listed so nobody trips on them; none blocks normal use.

#	What	Where	Effect
1	`core.route_permission` is never seeded	`core/permissions.js`, migrations	The route-table gate allows everything. Access control is entirely the handler checks, so never skip them.
2	The `hrbp` role cannot be granted from the app	`core/employees.js` `PUT /:id/role` accepts `employee, manager, hod, hr, hr_ops, admin`; `DirectoryPage.jsx` lists the same	`HrbpAdminPage.jsx` tells HR to "give somebody the hrbp role on the Employees page", which is not possible. Today it needs SQL on `core.user_roles`.
3	`engagement_admin` for HRBPs is granted only for tenants that existed when migration 073 ran	`migrations/073`, not in a boot seed	On a fresh install HRBPs rely on the gateway lending it, which works for `/pms` routes
4	Ten server test files use the database without a skip guard	`access-guards`, `career-horizon`, `connect-questions`, `external-review-import`, `hrbp-gateway`, `hrbp-self-service`, `rnr-scenarios`, `self-chase-7-days`, `survey-results-export`, `technical-skill-tiers`	Without `DATABASE_URL` they fail instead of skipping
5	The external-review import reads every file as CSV	`modules/engagement/index.js` (`detectFormat` called with a name instead of the file; the Excel parse not awaited)	Uploading an `.xlsx` external review fails; CSV works
6	Module boundary breaches	`people` and `agentic` import `performance/active-cycle` and `phase-machine`; `people`, `engagement` import migration files; `agentic` reads and writes other schemas directly; `core/employees.js` requires performance files	Works, but changes in one module can break another silently. Prefer exported interfaces in new code.
7	No global error handler, no rate limiting (except forgot password), CORS open to all origins	`server/index.js`	Behind nginx on one origin this is acceptable; revisit for a shared deployment
8	No global 401 handling in the browser	`frontend/src/utils/api.jsx`	An expired token shows the server's error on the page instead of returning to sign-in
9	`agentic.drafts` is created at runtime, not by a migration	`core/ai.js` `ensureTable()`	The table appears on the first AI call
10	Leftover tables	`pms.pulse_checks` (feature removed), `people.career_matrix` (unused), `rnr.band_levels` (unread)	Safe to ignore; don't build on them
11	The `agentic/index.js` header comment is out of date	It says five features and no writes to `pms`	There are about 20 endpoints, and parameter analysis writes `pms.parameter_ai_analyses`
12	`tools/` holds only sample files	`core/employees.js` header mentions a standalone tool there	The tool is not in this repo
13	The KRA bulk upload deletes and re-inserts a sheet's KRAs	`modules/performance/index.js` `POST /hr/kra-sheet/bulk-upload`	New KRA ids, so mid-year entries, growth-goal links and timesheet mappings on that sheet lose their KRA. Approved sheets are skipped. The screen editor (`writeKras`) keeps ids.
14	A typed manager overall is not range-checked	`PUT /team/evaluations/:employeeId` when `entries` is absent	The screens always send per-KRA entries, so the overall is computed; a direct API call could store any number
15	The HRBP tab has its own "active cycle" query	`modules/performance/hrbp.js` `activeCycle`	It does not skip `cancelled` cycles or prefer started ones, unlike `active-cycle.js`
16	Growth-plan approval checks the manager stored on the plan, KRA approval checks the live manager	`canDecide` in `index.js` vs `approvals.js`	After a reporting-line change, only open records follow the new manager (`manager-handover.js`)
17	Two quarterly-connect reminders run side by side	the 90-day cadence (`connect-reminders.js`) and the calendar `quarterly_connect` rule (`reminders.js`)	Both are deduplicated by their own ledgers; a person can get both

22. Glossary

Term	Meaning
Cycle	One appraisal period (`pms.cycles`). Its `phase` drives what is open.
Phases	`draft → kra_open → mid_year_review → self_appraisal → manager_eval → hod_eval → calibration → publish → closed` . Screen labels differ: `kra_open` is "KRA Setting and Growth Planning", `self_appraisal` is "Annual Review".
KRA / KPI	Key Result Area (an objective) and its measure. Each person's KRAs, with weights adding to 100%, form their **KRA sheet**.
KRA library	HR's catalogue of KRAs by department and role that employees pick from
Parameter	A group heading over KRAs on the sheet
Connect	A logged 1:1 conversation, signed off by the other person
MoM	Minutes of meeting, kept with connects and meetings
HOD	Head of department (`core.department_heads`); reviews after the manager
HRBP	HR business partner; HR's screens limited to a remit (`core.hrbp_scope`)
Calibration	HR's adjustment of ratings across the company; each change needs a reason
Kitty	The increment budget used in calibration and increment simulation
Publish	Writes final ratings to `pms.employee_performance_history` ; the employee then sees them
9-box	Performance × potential grid
Super 50	The top-talent watchlist and its benefits
PIP	Performance Improvement Plan with weekly entries and gates
RnR	Rewards & Recognition: awards, nominations, the HOD → HRBP → HR chain
Mirror	`core.employees` , loaded from the client's HR export, not edited as an HRMS
Remit	The set of people an HRBP covers (by location or HOD)
Draft	AI output, always labelled and stored with its input